Introduzione al linguaggio RISC-V

R. Cantoro, M. Rebaudengo,
M. Sonza Reorda, L. Sterpone

Politecnico di Torino
Dip. di Automatica e Informatica



I processori: cosa sono

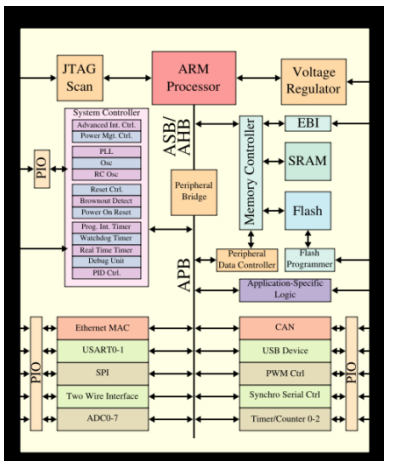
Il processore è il modulo principale di un sistema a processore.

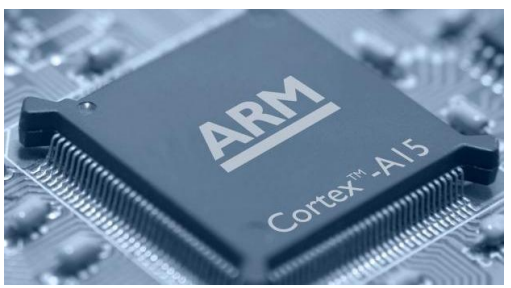
Un processore è un modulo che può corrispondere

- **a una parte (denominata *core*) di un circuito integrato più complesso (SoC o microcontrollore)**
- **a un circuito integrato a se stante.**

Esistono numerose tipologie di processori in termini di complessità, potenza di calcolo, architettura, ecc.

SoC





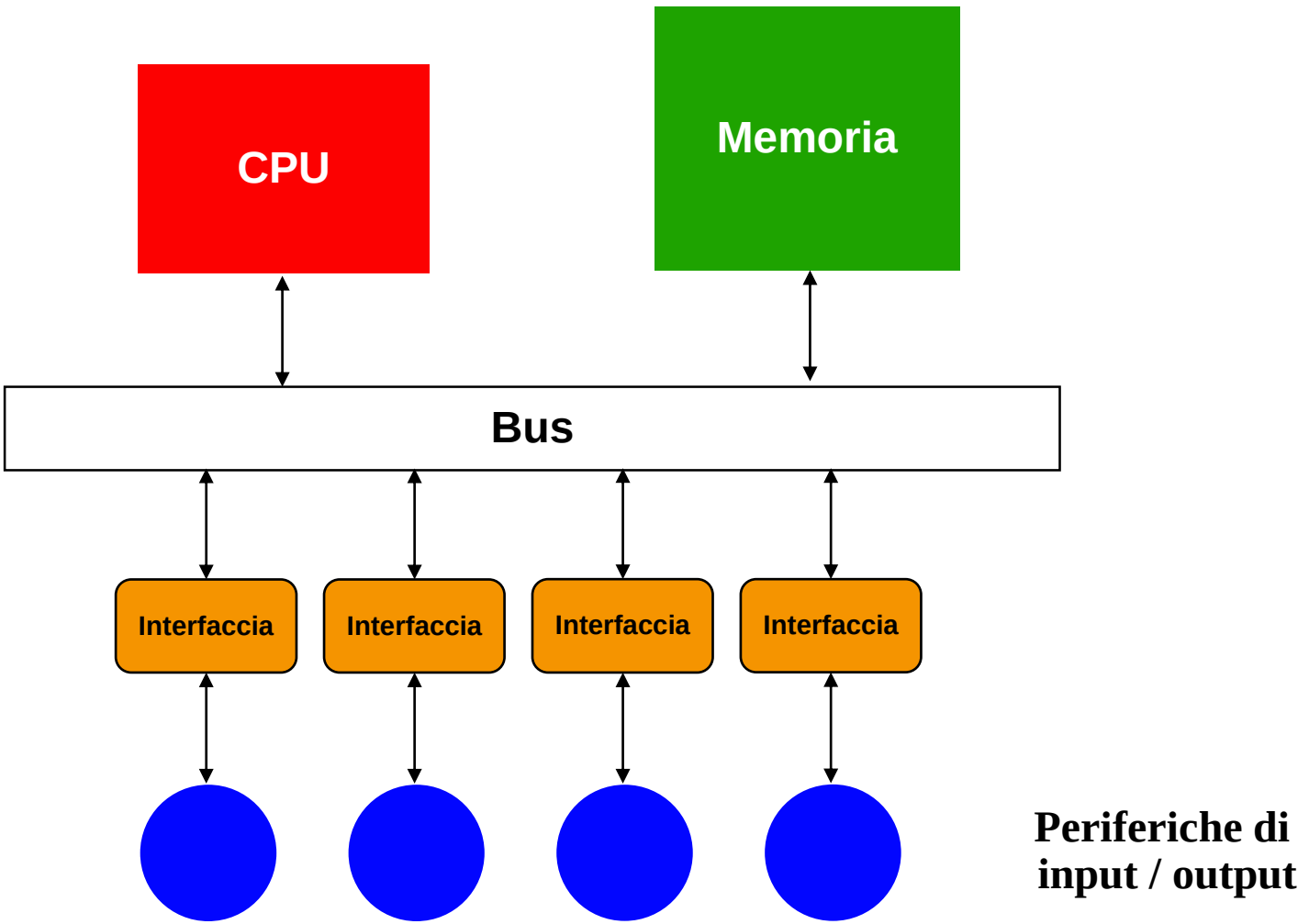
I processori: cosa fanno

Un processore è un dispositivo che compie 2 tipi di operazioni:

- **esegue istruzioni (il cui codice sta in memoria)**
- **interagisce con il mondo esterno (attraverso opportune interfacce).**

Le istruzioni che ciascun processore può eseguire sono quelle presenti nella sua ISA (istruzioni macchina).

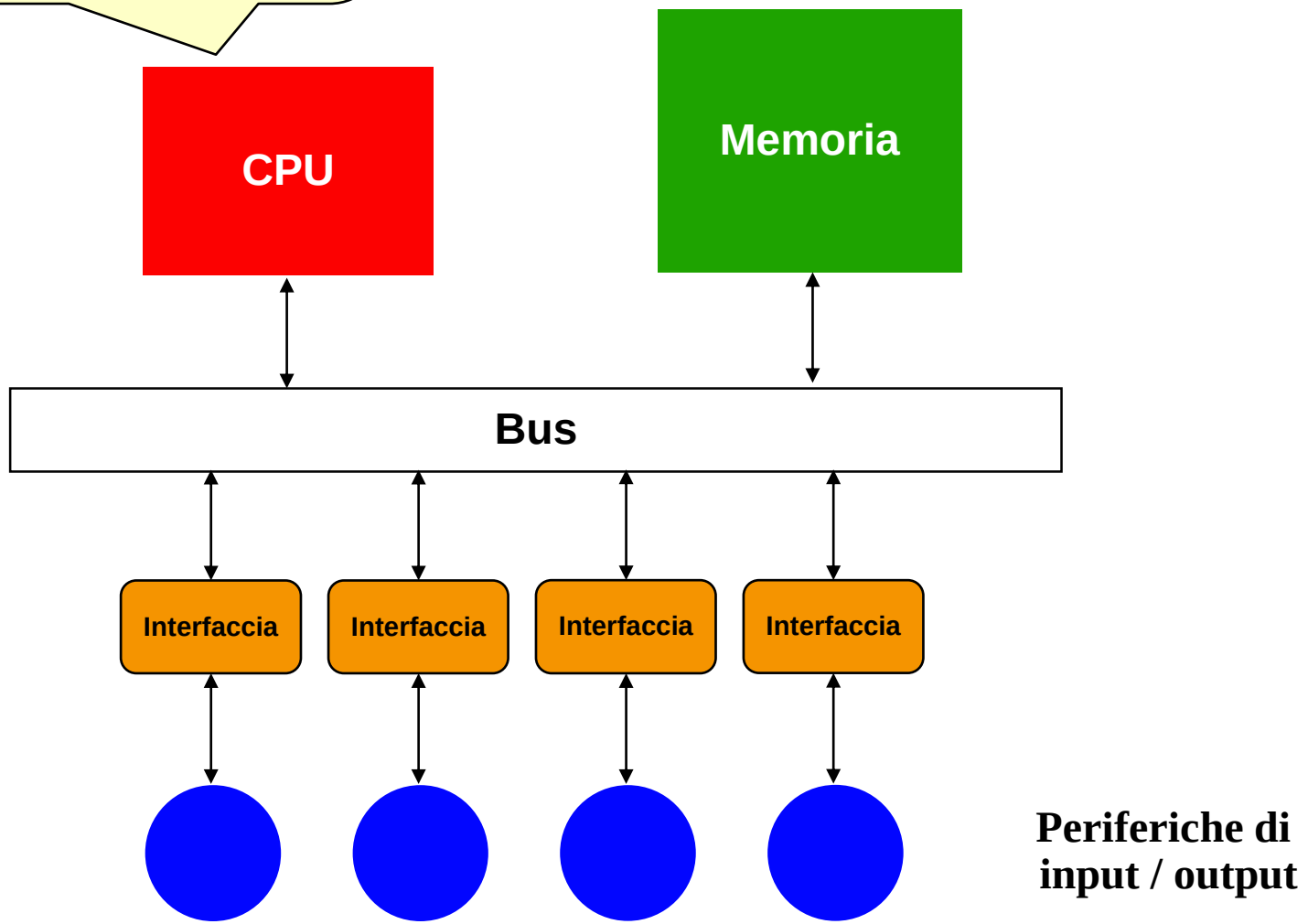
Sistema a processore



a proc

Esegue le istruzioni (memorizzate nella memoria) accedendo ai dati (memorizzati nella memoria)

Memorizza istruzioni e dati



Periferiche di input / output

Istruzioni

L’esecuzione di ciascuna istruzione si compone di 2 fasi:

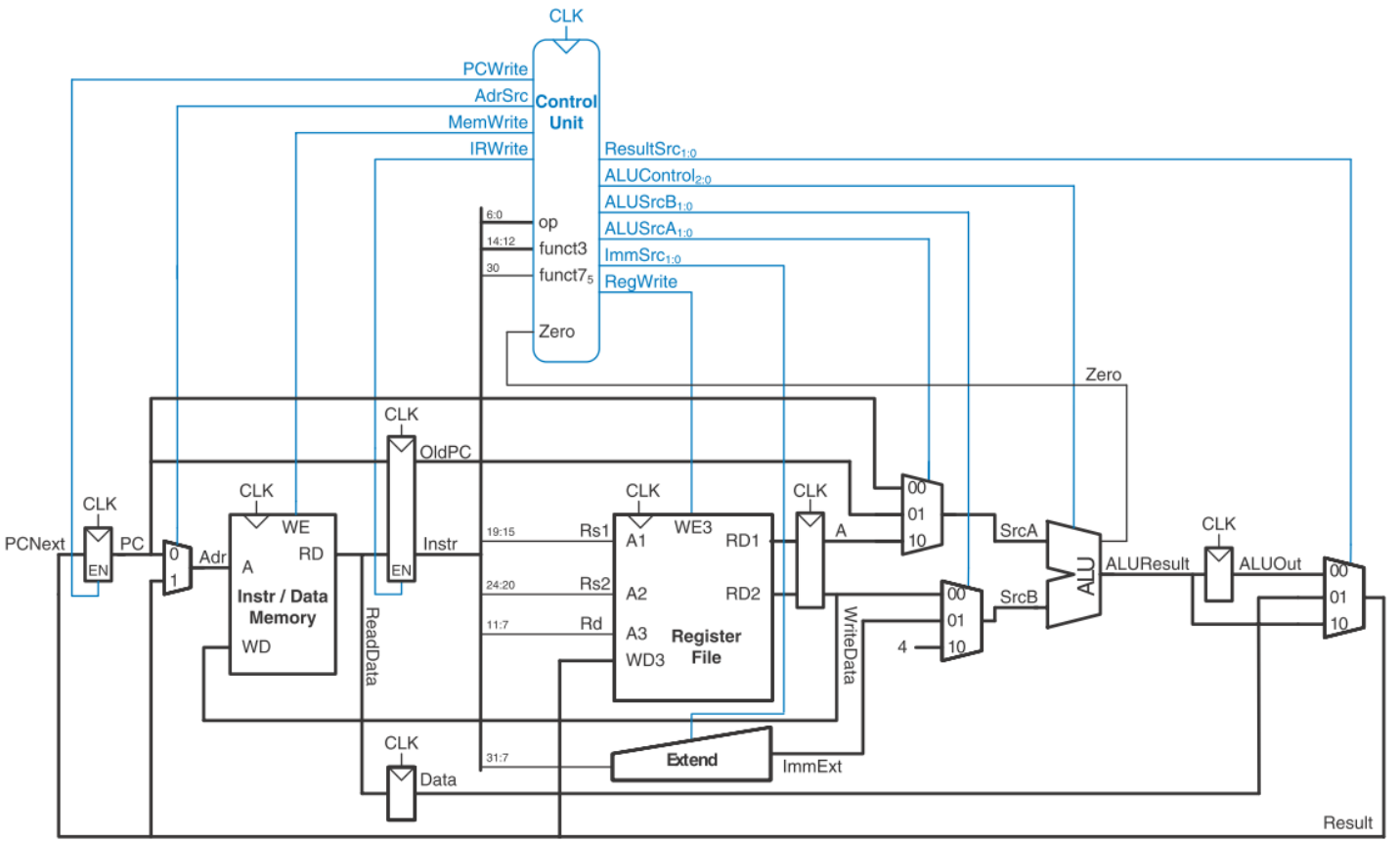
- ***fetch*: il codice dell’istruzione viene letto dalla memoria**
- ***execute*: il codice viene prima decodificato, poi eseguito. Questo comporta normalmente l’accesso ad uno o più operandi, l’esecuzione di un’operazione su di essi, la scrittura del risultato.**

La combinazione delle due fasi si dice *ciclo di istruzione*.

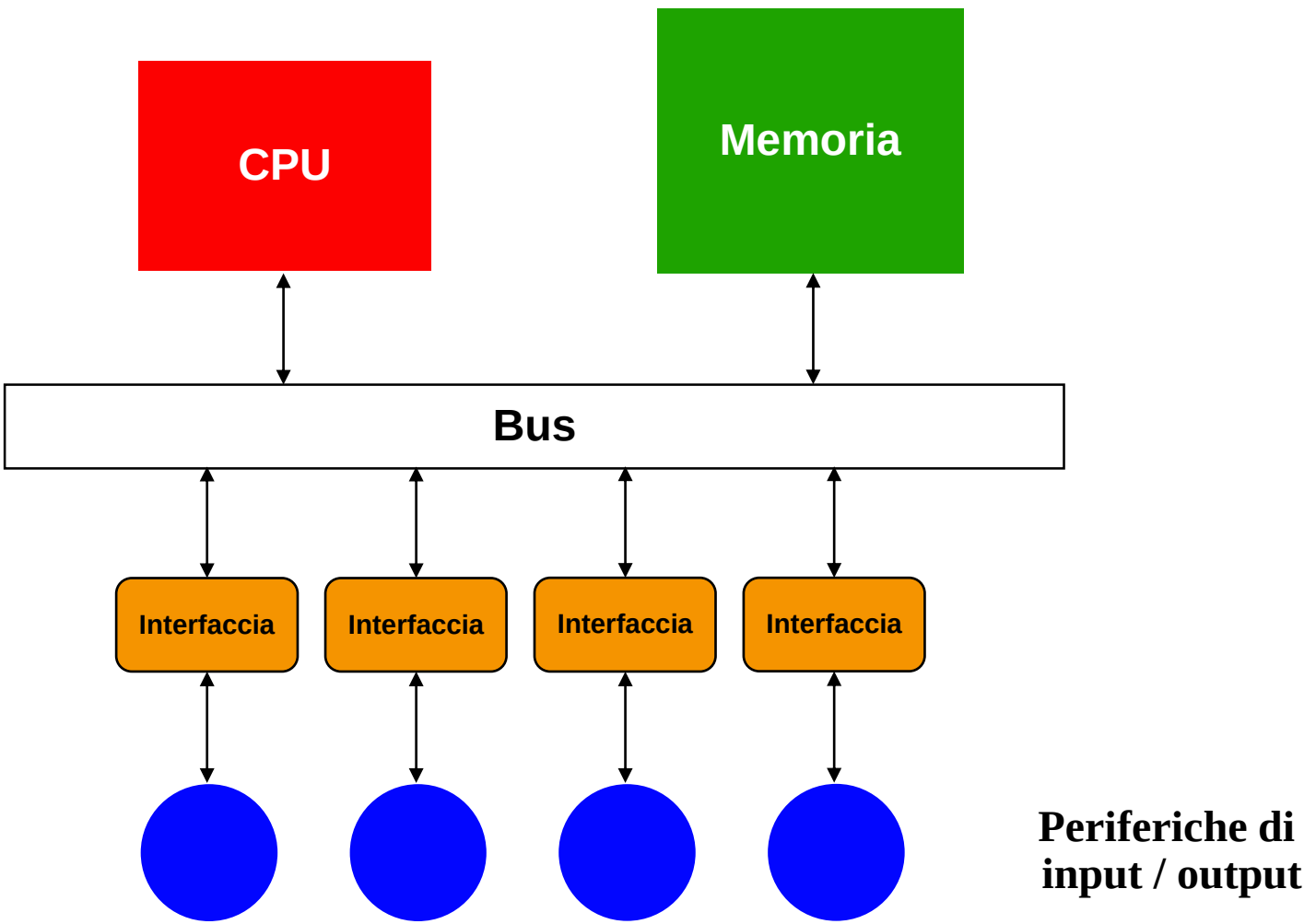
Registri

- **Il tempo per accedere alla memoria è normalmente superiore al tempo necessario alla CPU per processare i dati**
- **L'accesso alla memoria rappresenta quindi un *collo di bottiglia* per le prestazioni delle CPU**
- **Per questa ragione all'interno della CPU sono presenti alcune celle di memoria particolarmente veloci, note come *registri***
- **Ove possibile le operazioni vengono svolte utilizzando i registri per contenere operandi e risultato.**

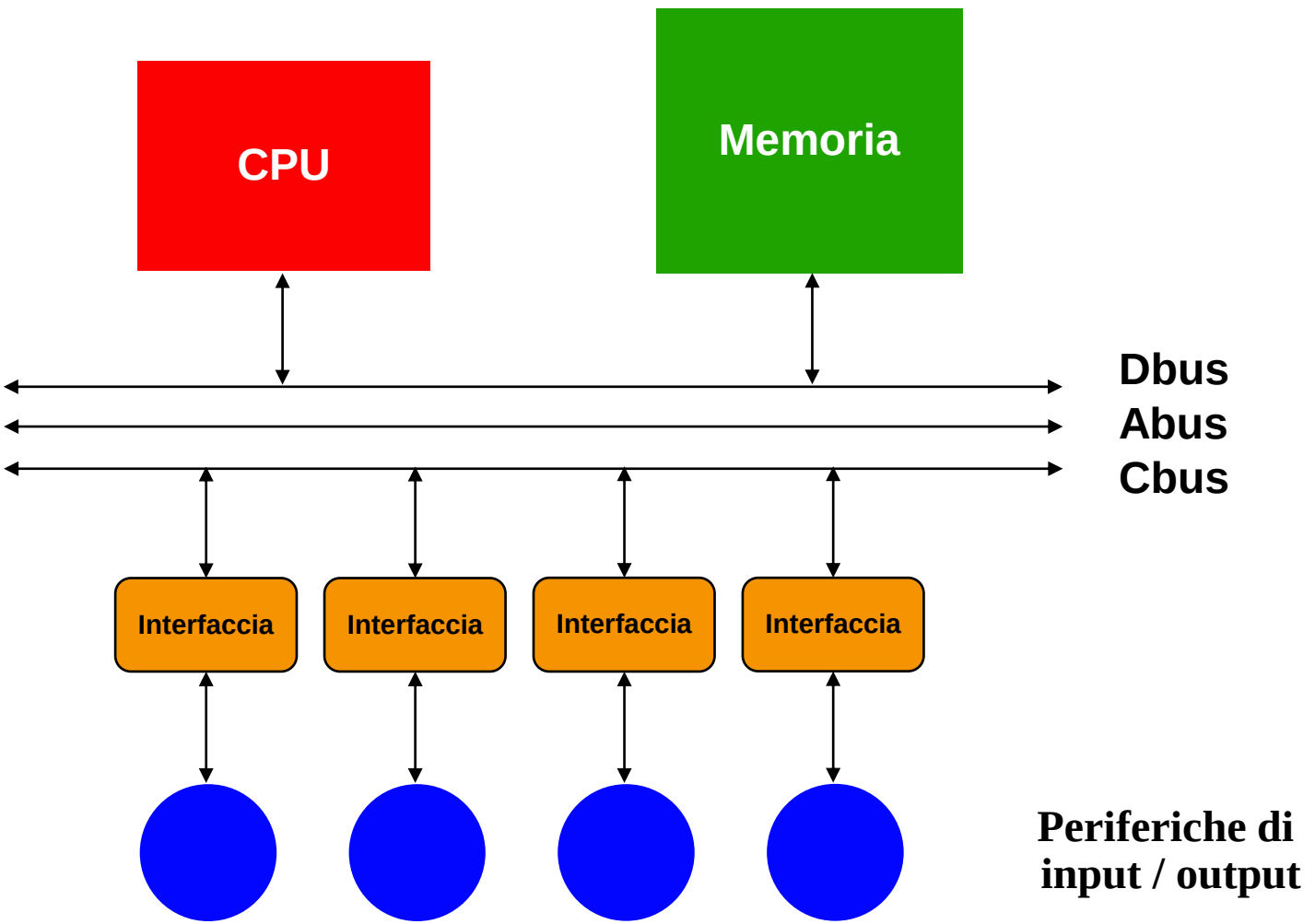
Possibile modello architetturale della CPU RISC-V



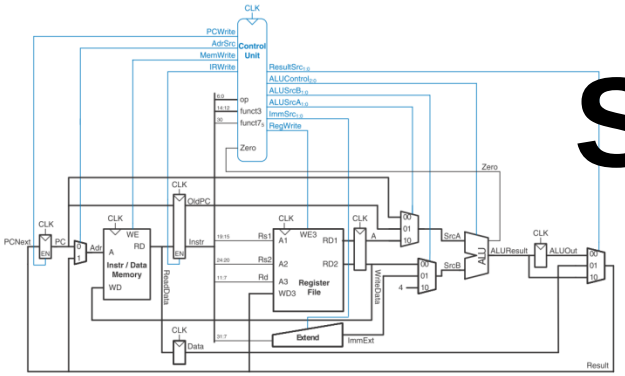
Sistema a processore

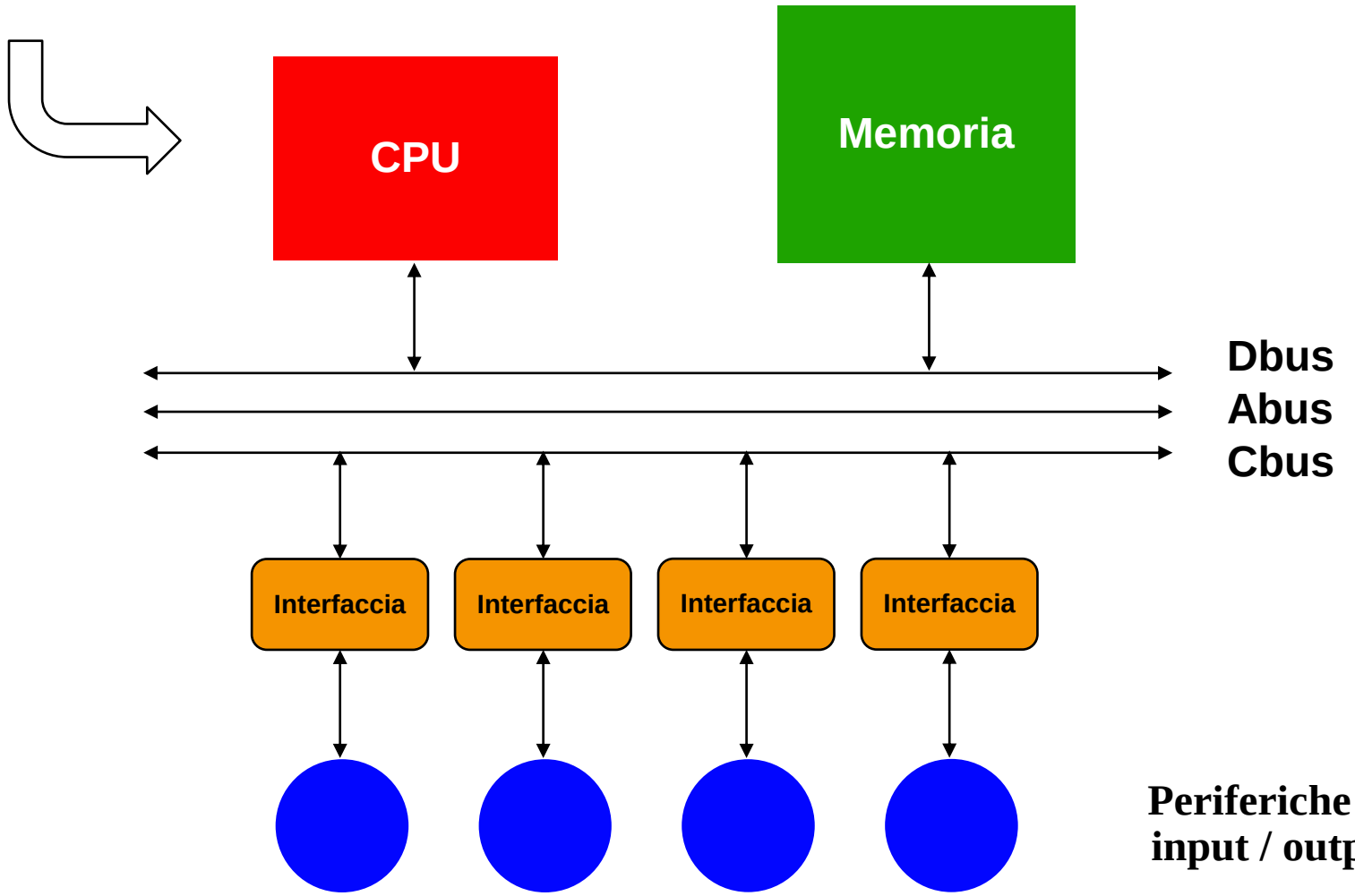


Sistema RISC-V



Sistema RISC-V





Ciclo di vita di un programma



Utente

Editor

Codice sorgente

Assemblatore

Codice oggetto

Linker

Codice eseguibile

Loader

Codice macchina

CPU





Assembly

- **Rappresentazione simbolica del linguaggio macchina**
- **Vero e proprio linguaggio di programmazione**
- **Più comprensibile del linguaggio macchina in quanto utilizza simboli invece che sequenze di bit**
- **Rispetto ai linguaggi ad alto livello...**
  - **Assembly è il linguaggio in cui è espresso il prodotto della compilazione di codice in linguaggi ad alto livello**
  - **Assembly fornisce limitate forme di controllo del flusso**
  - **Assembly non prevede strutture dati articolate**

Assembly

- **VANTAGGI:**
  - **Visibilità diretta dell'hardware**
  - **Massimo sfruttamento delle potenzialità HW della macchina**
  - **Ottimizzazione (potenziale) delle prestazioni**
- **SVANTAGGI:**
  - **Ridotta portabilità dei programmi**
  - **Maggior costo di sviluppo**
  - **Maggiore lunghezza e difficoltà di comprensione**

C, Python, and Assembly

C:

```
...
int sum = 0;
    for (i=0; i<=100; i=i+1)
      sum = sum + i;
...
```

Python:

```
...
  sum = 0
  for i in range(101):
      sum = sum + i
...
```

Assembly:

```
…
        addi t0, x0, 100
        addi t1, x0, 0
        addi a0, x0, 0

etich:  add a0, a0, t1
        addi t1, t1, 1
        bge  t0, t1, etich
…
```

Scrittura di un valore in un registro

```
a = 0;
```

```
#  scrive 0 in t0

.text
main:
   and t0, x0, x0

   li a7, 10  # termina il programma
   ecall
```

Somma di due valori

```
int     opd1=0x5678,
        opd2=0x12340000,
        risult;
```

```
risult = opd1 + opd2;
```

```
# legge 2 valori, li somma e li riscrive in memoria
.data
opd1:       .word 0x00005678
opd2:       .word 0x12340000
risult:     .zero 4
.text
main:
            la      t0, opd1
            lw      t1, 0(t0)           #carica opd1 in t1
            la      t0, opd2
            lw      t2, 0(t0)           #carica opd2 in t2
            add     t3, t1, t2          #somma
            la      t0, risult
            sw      t3, 0(t0)           #scrive in risult

            li a7, 10
            ecall
```

Somma degli elementi di un vettore (I)

C:

```
int    vett[5]={
          5,7,3,4,3},
       risult,
       tmp;
```

Python:

```
vett = [5,7,3,4,3]
```

```
tmp = 0;
tmp += vett[0];
tmp += vett[1];
tmp += vett[2];
tmp += vett[3];
tmp += vett[4];
result = tmp;
```

```
risult = sum(vett)
```

```
#  somma gli elementi di un vettore
.data
vett:              .word 5, 7, 3, 4, 3
risult:            .word 0

.text
main:
        la   t3, vett     # indirizzo di vett
        add  t4, x0, x0   # scrive 0 in t4
        lw   t0, 0(t3)    # carica in t0 il primo elemento
        add  t4, t4, t0   # somma t0 a t4
        addi t3, t3, 4    # aggiorna t3
        lw   t0, 0(t3)    # carica in t0 il secondo elemento
        add  t4, t4, t0   # somma t0 a t4
        addi t3, t3, 4    # aggiorna t3
        lw   t0, 0(t3)    # carica in t0 il terzo elemento
        add  t4, t4, t0   # somma t0 a t4
        addi t3, t3, 4    # aggiorna t3
        lw   t0, 0(t3)    # carica in t0 il quarto elemento
        add  t4, t4, t0   # somma t0 a t4
        addi t3, t3, 4    # aggiorna t3
        lw   t0, 0(t3)    # carica in t0 il quinto elemento
        add  t4, t4, t0   # somma t0 a t4
        la   t3, risult   # indirizzo di risult
        sw   t4, 0(t3)    # scrivi risultato in memoria

        li a7, 10
        ecall
```

Somma degli elementi di un vettore (II)

```
int    vett[5],
       risult,
       tmp;

for(i=0; i<5; i++)
       tmp += vett[i];

risult = tmp;
```

```
#  somma gli elementi di un vettore
.data
vett:   .word 5, 7, 3, 4, 3
risult: .word 0
.text
main:
        la   t1, vett
        add  t3, x0, x0        # scrive 0 in t3
        add  t4, x0, x0        # scrive 0 in t4
        addi t5, x0, 5         # scrive 5 in t5
etich:  lw   t0, 0(t1)         # carica in t0 un elemento
        add  t4, t4, t0        # somma t0 a t4
        addi t3, t3, 1         # aggiorna t3
        addi t1, t1, 4         # aggiorna t4
        bne  t3, t5, etich     # ripeti

        la   t1, risult
        sw   t4, 0(t1) # scrive il risultato in memoria

        li a7, 10
        ecall
```

Lettura e visualizzazione di un vettore di caratteri

```
# legge 5 caratteri da tastiera, li memorizza in un vettore e li visualizza in ordine inverso
.data
vett:     .zero 6   # il sesto carattere è il new line

          .text
main:
          addi a7, x0, 63
          add  a0, x0, x0     # file descriptor 0: stdin
          la   a1, vett
          addi a2, x0, 6      # max numero di caratteri da leggere
          ecall               # legge un carattere

          addi t1, a0, -1     # numero di caratteri letti meno uno (per togliere il new line)
          la   t2, vett
          add  t2, t2, t1
          addi t2, t2, -1

ciclo:    lb   a0, 0(t2)
          addi a7, x0, 11
          ecall               # visualizza un carattere

          addi t1, t1, -1     # aggiorna t2
          addi t2, t2, -1     # aggiorna t2
          bne  t1, x0, ciclo

          li   a7, 10
          ecall
```

```
char    vett[5];
```

```
for(i=0; i<5; i++)
        vett[i] = getchar();
```

```
for(i=4; i>=0; i--)
        putchar(vett[i]);
```